Curso: Sistemas de Informação	
Disciplina: Mapemento Objeto Relacional	**Período: 4**
Professor: Luiz Gustavo Dias	**Tipo:** Material Complementar
Objetivo: Relacionamentos	

RELACIONAMENTOS

Em aplicações que utilizam Java, é comum que as informações sejam distribuídas entre diferentes entidades, cada uma responsável por representar um conceito do mundo real. Em um sistema acadêmico, por exemplo, podemos ter as entidades Aluno, Curso, Professor e Disciplina. Em um sistema comercial, podemos encontrar Cliente, Pedido, Produto e Categoria.

Embora cada entidade represente um conceito específico, elas não existem necessariamente de forma isolada. Um aluno está matriculado em um curso, um professor ministra disciplinas e um cliente realiza pedidos que contêm produtos. Essas relações fazem parte das regras de negócio da aplicação e precisam ser representadas tanto no código Java quanto no banco de dados.

Até este momento, já trabalhamos com o uso do JPA para persistir objetos, inserir registros, atualizar informações e excluir dados no MySQL. Também utilizamos o Hibernate como implementação do JPA e estudamos a importância do gerenciamento adequado das transações. O próximo passo consiste em ampliar essa compreensão, aprendendo a representar relacionamentos entre entidades distintas.

O objetivo é compreender como o JPA permite estabelecer essas relações, quais são os tipos de associação disponíveis, como elas são mapeadas para o banco de dados e como utilizá-las na prática, mantendo a consistência das informações.

TIPOS DE RELACIONAMENTOS ENTRE ENTIDADES

Relacionamento um para um — @OneToOne

O relacionamento um para um ocorre quando uma entidade está associada a, no máximo, uma instância de outra entidade, e vice-versa.

Imagine um sistema acadêmico no qual cada aluno possui um único registro de endereço principal, e cada registro de endereço pertence a um único aluno. Nesse cenário, podemos representar a associação entre Aluno e Endereco utilizando @OneToOne.

A implementação poderia ser estruturada da seguinte maneira:

```java
package objects;

import javax.persistence.*;

@Entity
public class Aluno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    @Column
    private String nome;

    // Relacionamento com a entidade Endereco
    // Um aluno possui um endereco
    @OneToOne
    @JoinColumn(name = "id_endereco")
    private Endereco endereco;
```

A anotação @OneToOne informa ao JPA que existe uma associação um para um entre as entidades. Já @JoinColumn(name = "id_endereco") indica o nome da coluna que armazenará a chave estrangeira na tabela correspondente a Aluno.

Nesse exemplo, a coluna id_endereco deverá referenciar a chave primária da tabela endereco. Para garantir a cardinalidade um para um também no banco de dados, a coluna que contém a chave estrangeira deve possuir uma restrição de unicidade, normalmente criada pelo próprio mapeamento ou definida no esquema do banco.

Observe que o relacionamento não significa necessariamente que as duas entidades precisam ser criadas ou excluídas simultaneamente. Esse comportamento depende das regras da aplicação e das configurações de persistência.

Relacionamento muitos para um — @ManyToOne

O relacionamento muitos para um ocorre quando várias instâncias de uma entidade podem estar associadas a uma única instância de outra entidade.

Retomando o exemplo acadêmico, vários alunos podem estar matriculados no mesmo curso. Portanto, do ponto de vista de Aluno, o relacionamento com Curso é muitos para um.

```
21
22      // Relacionamento com a entidade Curso
23      // Muitos alunos possuem matrícula em um curso
24      @ManyToOne
25      @JoinColumn(name = "id_curso")
26      private Curso curso;
```

Nesse caso, a entidade Aluno possui um atributo do tipo Curso. Esse atributo representa o objeto ao qual o aluno está associado.

A anotação @ManyToOne informa ao JPA que vários alunos podem apontar para o mesmo curso. A anotação @JoinColumn identifica a coluna da tabela aluno responsável por armazenar essa associação.

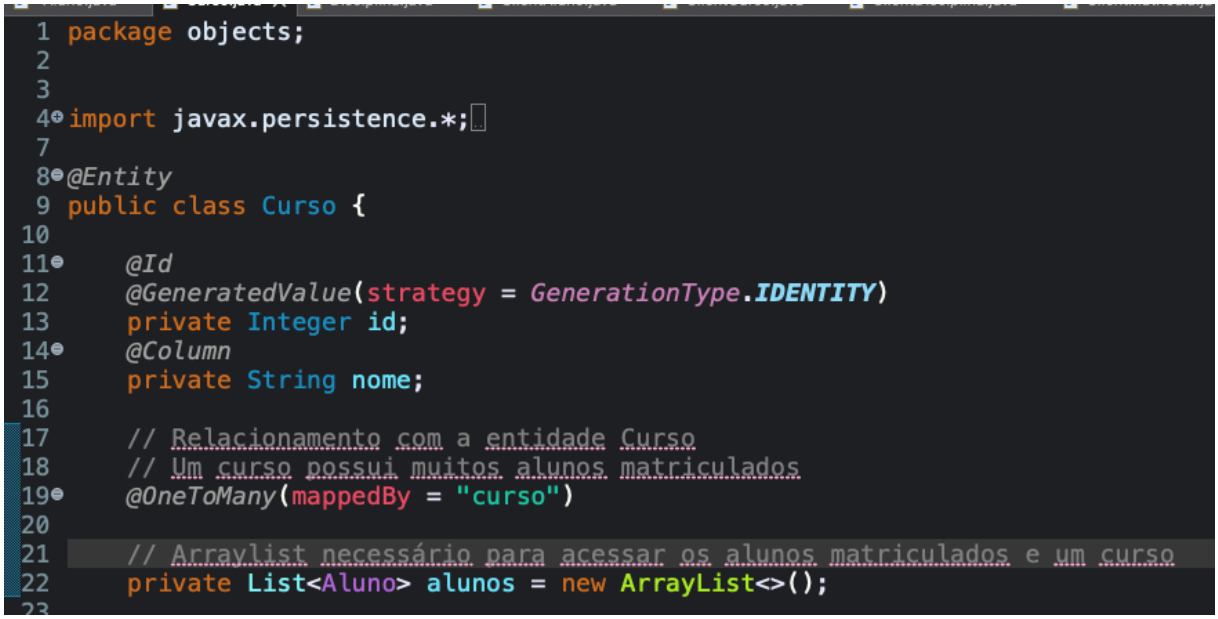
Uma característica importante desse relacionamento é que a chave estrangeira normalmente fica na tabela que representa o lado muitos. Assim, cada aluno armazena a referência ao curso ao qual está associado, sem que seja necessário repetir os dados completos do curso em cada registro.

Relacionamento um para muitos — @OneToMany

O relacionamento um para muitos representa o outro lado da associação anterior: uma única instância de uma entidade pode estar associada a várias instâncias de outra.

No exemplo acadêmico, um curso pode possuir vários alunos. Portanto, do ponto de vista de Curso, temos um relacionamento um para muitos com Aluno.

Para representar essa associação, podemos utilizar uma coleção de objetos:



```java
package objects;


import javax.persistence.*;

@Entity
public class Curso {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    @Column
    private String nome;

    // Relacionamento com a entidade Curso
    // Um curso possui muitos alunos matriculados
    @OneToMany(mappedBy = "curso")

    // Arraylist necessário para acessar os alunos matriculados e um curso
    private List<Aluno> alunos = new ArrayList<>();
```

O atributo alunos representa uma coleção de objetos da classe Aluno. Dessa forma, um objeto Curso pode disponibilizar uma lista com os alunos associados a ele.

O detalhe mais importante desse exemplo é a utilização de mappedBy = "curso". Esse parâmetro informa que o relacionamento já está mapeado pelo atributo curso da entidade Aluno.

Em outras palavras, o JPA compreende que a associação é controlada pelo atributo curso, que utiliza @ManyToOne e referencia a coluna id_curso.

Uma regra fundamental: em um relacionamento bidirecional entre Curso e Aluno, o lado @ManyToOne normalmente é o responsável pela chave estrangeira. O lado @OneToMany, quando utiliza mappedBy, representa o lado inverso da mesma associação.

Isso evita que o mapeamento tente administrar a mesma relação como se fossem duas associações independentes.

Também é importante lembrar que @OneToMany não significa que os alunos serão automaticamente carregados sempre que um curso for consultado. O comportamento de carregamento depende das configurações de busca da associação, assunto que será aprofundado adiante.

Relacionamento muitos para muitos — @ManyToMany

O relacionamento muitos para muitos ocorre quando várias instâncias de uma entidade podem estar associadas a várias instâncias de outra.

Considere um sistema no qual alunos podem se matricular em várias disciplinas e cada disciplina pode possuir vários alunos. Nesse cenário, existe um relacionamento muitos para muitos entre Aluno e Disciplina.

No banco de dados relacional, essa associação costuma ser implementada por meio de uma terceira tabela, denominada tabela associativa ou tabela de junção. Ela armazena as chaves estrangeiras das duas tabelas relacionadas.

No JPA, esse relacionamento pode ser mapeado com @ManyToMany e @JoinTable:

```
28      // Relacionamento Proprietário com a entidade Disciplina
29      // Muitos alunos possuem matrícula em muitas disciplinas
30      @ManyToMany
31      @JoinTable(
32          name = "aluno_disciplina",
33          joinColumns = @JoinColumn(name = "id_aluno"),
34          inverseJoinColumns = @JoinColumn(name = "id_disciplina")
35      )
36
37      // Arraylist para acessar as disciplinas nas quais o aluno está matriculado
38      private List<Disciplina> disciplinas = new ArrayList<>();
```

A anotação @JoinTable informa que a associação será representada pela tabela matricula. O parâmetro joinColumns identifica a coluna relacionada à entidade proprietária do mapeamento, enquanto inverseJoinColumns identifica a coluna que referencia a outra entidade.

Embora @ManyToMany seja útil em situações simples, é necessário avaliar se a tabela associativa possui atributos próprios. Se uma matrícula precisar armazenar data, nota, frequência ou situação, por exemplo, a tabela deixa de representar apenas uma associação e passa a possuir informações relevantes para o negócio.

Nesse caso, uma solução mais adequada costuma ser criar uma entidade específica, como Matricula, relacionando-a a Aluno e Disciplina. Essa abordagem permite representar explicitamente os dados da matrícula e aplicar regras de negócio próprias.

RELACIONAMENTOS UNIDIRECIONAIS E BIDIRECIONAIS

Além da cardinalidade, os relacionamentos podem ser classificados conforme a direção pela qual são acessados no modelo de objetos.

Um relacionamento é unidirecional quando apenas uma entidade mantém a referência à outra. Por exemplo, se Aluno possui um atributo curso, mas Curso não possui uma coleção de alunos, podemos consultar o curso a partir do aluno, mas não percorrer diretamente a associação no sentido contrário por meio de um atributo Java.

Um relacionamento é bidirecional quando ambas as entidades possuem atributos que permitem navegar pela associação. No exemplo anterior, Aluno referencia Curso, enquanto Curso mantém uma coleção de Aluno.

A escolha entre essas duas abordagens depende das necessidades da aplicação. Não é obrigatório mapear todas as relações nos dois sentidos. Se o sistema precisa apenas consultar o curso de cada aluno, um relacionamento unidirecional pode ser suficiente.

Relacionamentos bidirecionais oferecem maior flexibilidade de navegação, mas também exigem maior cuidado para manter os dois lados do relacionamento consistentes em memória.